

《现代C++编程实现》电子版: <https://leanpub.com/c01>

第7章 函数模板

function template

YouTube频道: [hwdong](https://www.youtube.com/hwdong)

博客: hwdong-net.github.io

B站: hw-dong

函数模板(function template)

- 函数模板是C++中实现泛型编程的重要工具，允许我们编写通用的代码，适用于多种数据类型，而无需重复编写几乎完全相同的函数。
- C++ 标准库中广泛采用了函数模板，以支持任意数据类型（包括将来可能定义的用户类型）。

函数模板(function template)

- 7.1 为什么需要函数模板？
- 7.2 函数模板的定义与实例化
- 7.3 显式指定模板参数与自动推断模板参数
- 7.4 非类型模板参数、7.5 模板模板参数
- 7.6 模板参数的默认值、7.7 返回类型推断
- 7.8 函数模板与重载
- 7.9 模板专门化
- 7.10 可变模板参数

7.1 为什么需要函数模板?

- 排序

```
void sort(int arr[], int n) {  
    //...  
}
```

```
void sort(double arr[], int n) {  
    //...  
}  
void sort(Student arr[], int n) {  
    //...  
}
```

7.1 为什么需要函数模板？

- **代码重复**：针对不同类型的重载函数（如`sort(double arr[], int n)`、`sort(Student arr[], int n)` 和 `sort(int arr[],int n)`）的逻辑完全相同，只是数据类型不同。
- **扩展性差**：如果需要在支持更多类型（例如 `float`、`char`），必须针对新类型，编写新的函数。
- **维护困难**：如果算法逻辑需要修改，所有版本都必须修改，既单调又容易出错，维护成本高。

7.1 为什么需要函数模板？

- 交换两个值

```
void swap(int& a, int& b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

```
void swap(double& a, double& b) {  
    double temp = a;  
    a = b;  
    b = temp;  
}
```

```
void swap(std::string& a, std::string& b) {  
    std::string temp = a;  
    a = b;  
    b = temp;  
}
```

7.2 函数模板的定义与实例化

- 函数模板的定义

```
template <typename T> // 声明模板，T 是类型参数  
返回类型 函数名(参数列表) {  
    // 函数体  
}
```

- `template`：关键字，告诉编译器这是一个模板。
- `typename T`：定义一个**类型参数** T，可以是任意类型（int、double、用户自定义类型等）。
- `template <typename T>`：称为**模板头**。表示这是一个模板，而 `T` 为模板类型参数，代表任意数据类型。
- 返回类型 函数名(参数列表)：类似普通的函数声明，只不过其中的参数类型是模板（类型）参数。
- 函数体：使用 T 作为数据类型来编写通用逻辑。

7.2 函数模板的定义与实例化

- 函数模板的定义

```
template <typename T> // 声明模板，T 是类型参数  
返回类型 函数名(参数列表) {  
    // 函数体  
}
```

```
template <typename T>  
void swap(T& a, T& b) {  
    T temp = a;  
    a = b;  
    b = temp;  
}
```


7.2 函数模板的定义与实例化

- 函数模板的定义

```
template <typename T> // 声明模板，T 是类型参数  
返回类型 函数名(参数列表) {  
    // 函数体  
}
```

```
template <typename T>  
T Max(T a, T b) {  
    return a > b ? a : b;  
}
```

7.2 函数模板的定义与实例化

- 函数模板的实例化：

函数模板本身并不是一个具体的函数，而是用来生成具体函数的一种蓝图。只有在调用函数模板时，编译器才会根据传入的实参，生成对应类型的函数实例。这一过程称为**函数模板的实例化**。

```
int main() {  
    std::cout << Max<int>(3, 5) << std::endl;  
    std::cout << Max(3.5, 4.5) << std::endl;  
}
```

7.2 函数模板的定义与实例化

- 函数模板的实例化：

实例化时，可以通过**显式指定模板参数**，也可以让编译器根据实参类型**自动推断模板参数**。

```
int main() {  
    std::cout << Max<int>(3, 5) << std::endl;  
    std::cout << Max(3.5, 4.5) << std::endl;  
}
```

7.2 函数模板的定义与实例化

- 显示指定模板参数

编译器遇到 `Max<int>(3, 5)` 时，会实例化出如下函数：

```
int Max(int a, int b) {  
    return a > b ? a : b;  
}
```

7.2 函数模板的定义与实例化

- 自动推断模板参数

编译器遇到 `Max(3.5, 4.5)` 时，会实例化出如下函数：

```
double Max(double a, double b) {  
    return a > b ? a : b;  
}
```

7.3 显式指定模板参数与自动推断模板参数

- **显式指定模板参数**：在调用函数模板时，明确写出模板参数。
- **自动推断模板参数**：让编译器根据函数实参类型，自动推断模板参数的类型。

```
Max<int>(3, 5)
```

```
Max(3.5, 4.5)
```

7.3 显式指定模板参数与自动推断模板参数

- 自动推断，或者推断无法满足需求，可以显式提供模板参数：

```
cout<<Max(3,4.5);
```

7.3 显式指定模板参数与自动推断模板参数

- 自动推断不能保留**引用**或**const**

```
#include <iostream>
#include <typeinfo>

template <typename T>
void func(T x) {
    std::cout << "T = " << typeid(T).name() << std::endl;
}

int main() {
    const int a = 42;
    const int& ref = a;

    func(ref); // 编译器自动推断 T
}
```

输出:

T = int

7.3 显式指定模板参数与自动推断模板参数

- 自动推断不能保留**引用**或**const**

```
#include <iostream>
#include <typeinfo>

template <typename T>
void func(T x) {
    std::cout << "T = " << typeid(T).name() << std::endl;
}

int main() {
    const int a = 42;
    const int& ref = a;

    func<const int&>(ref);
}
```

输出结果变为：

```
T = int const&
```

7.4 非类型模板参数

- 函数模板中包含两个非类型模板参数 lower 和 upper 用于表示数组的区间:

```
template <typename T, int lower, int upper>
T average(const T arr[]) {
    T ret = arr[lower];
    for (int i = lower + 1; i <= upper; i++)
        ret += arr[i];
    return ret;
}
```

```
template <typename T, int lower, int upper>
T average(const T arr[]) {
    T ret = arr[lower];
    for (int i = lower + 1; i <= upper; i++)
        ret += arr[i];
    return ret;
}
```

使用示例：

```
int main() {
    int arr[] = { 1, 2, 3, 4, 5 };
    std::cout << average<int, 0, 4>(arr) << '\n';
    std::cout << average<int, 1, 3>(arr) << '\n';

    double arr2[] = { 1.2, 2.2, 3.4, 4.7, 5.8 };
    std::cout << average<double, 1, 3>(arr2) << '\n';
}
```

- 也可以将类型模板参数放在后面，让编译器根据函数实参自动推断，

```
template <int lower, int upper, typename T>
T average(const T arr[]) { /*...*/ }
```

```
int main() {
    int arr[] = { 1, 2, 3, 4, 5 };
    std::cout << average<0, 4>(arr) << '\n';
    std::cout << average<1, 3>(arr) << '\n';

    double arr2[] = { 1.2, 2.2, 3.4, 4.7, 5.8 };
    std::cout << average<1, 3>(arr2) << '\n';
}
```

- 在 C++17 中，还允许用 auto 声明非类型模板参数，使得模板定义更加简洁，例如：

```
template <auto value>  
void f() { }
```

```
f<10>(); // 自动推断 value 为 int 类型
```

- 非类型模板参数常用于编译期计算或元编程。auto 使得模板可以接受更广泛的值类型。例如：

```
#include <iostream>
using std::cout;

template <auto value>
constexpr auto compute_square() {
    return value * value; // 编译期计算
}

int main() {
    cout<<compute_square<5>();      // 输出: 25
    cout<<compute_square<2.5>();    // 输出: 6.25
    return 0;
}
```

意义：模板函数可以在编译期对任意类型的 `value` 执行计算，增强了代码的通用性。

```
#include <iostream>
using std::cout;

template <auto value>
constexpr auto compute_square() {
    return value * value; // 编译期计算
}

int main() {
    cout<<compute_square<5>();      // 输出： 25
    cout<<compute_square<2.5>();    // 输出： 6.25
    return 0;
}
```

- constexpr 常量模板。传统写法：

```
template <typename T, T value>
```

```
constexpr T TConstant = value;
```

```
constexpr auto MySuperConst = TConstant<int, 100>;
```


- constexpr 常量模板。使用 `auto` 非类型模板参数

```
template <auto value>
constexpr auto TConstant = value;

constexpr auto MySuperConst = TConstant<100>;
```

```
template <typename T, T value>
constexpr T TConstant = value;

constexpr auto MySuperConst = TConstant<int, 100>;
```

7.5 模板模板参数

- 有时候，模板的参数不仅是类型或数值，而是另一个模板。这种参数称为 **模板模板参数**。例如：

```
template <template <class> class X, class A>
void f(const X<A>& value) {
    // ...
}
```

7.6 模板参数的默认值

- 在定义函数模板时，可以为模板参数设置默认值。默认模板参数不仅适用于类型模板参数，也适用于非类型模板参数和模板模板参数

```
template <typename T = int, int e = 2>
T power(const T x) {
    T ret = x;
    for (int i = 1; i < e; i++)
        ret *= x;
    return ret;
}
```

```
template <typename T = int, int e = 2>
T power(const T x) {
    T ret = x;
    for (int i = 1; i < e; i++)
        ret *= x;
    return ret;
}

int main() {
    std::cout << power(3) << '\t';           // 使用默认 e = 2, T 由实参推断为 int
    std::cout << power(3.5) << '\t';          // T 推断为 double
    std::cout << power<double, 3>(3.5) << '\t'; // 显式指定 T 为 double, e = 3
}
```

- 默认模板参数与函数默认参数不同，模板参数的默认值可以放在任意位置

```
template <typename T = int, int e>
T power(const T x) {
    T ret = x;
    for (int i = 1; i < e; i++)
        ret *= x;
    return ret;
}
```

```
template <typename T = int, int e = 2>
```

```
T fun() {
```

```
    T ret = 0;
```

```
    for (int i = 1; i < e; i++)
```

```
        ret += 3.14;
```

```
    return ret;
```

```
}
```

```
int main() {
```

```
    std::cout << fun() << '\t';
```

// 都使用默认模板参数

```
    std::cout << fun<double>() << '\t';
```

// 指定 T 为 double, e 为默认值 2

```
    std::cout << fun<3>() << '\t';
```

// 错误: 不能用数值 3 实例化类型模板参数 T

```
    std::cout << fun<double, 3>() << '\n';
```

```
}
```

7.7 返回类型推断

- 在编写函数模板时，有时函数的返回类型依赖于传入参数的运算结果。

```
template <typename T1, typename T2>  
? add(T1 a, T2 b) {  
    return a + b;  
}
```

```
auto ret = add(2, 3);  
auto ret2 = add(2, 3.5);
```

7.7 返回类型推断

- 如何让编译器自动推导出正确的返回类型呢？

```
template <typename T1, typename T2>  
? add(T1 a, T2 b) {  
    return a + b;  
}
```

```
auto ret = add(2, 3);  
auto ret2 = add(2, 3.5);
```


7.7 返回类型推断

- **尾返回类型** (trailing return type) 语法

```
template <typename T1, typename T2>  
auto add(T1 a, T2 b) -> decltype(a + b) {  
    return a + b;  
}
```

7.7 返回类型推断

- 用 `decltype(auto)` 来推断返回类型：

```
template <typename T1, typename T2>
decltype(auto) add(T1 a, T2 b) {
    return a + b;
}
```

这里需要注意的是 `decltype(auto)` 和 `auto` 的区别：

- `auto` 总是返回值的拷贝（即使原始表达式是引用类型，返回值也会丢失引用）。
- `decltype(auto)` 可以保留原始表达式的类型属性（例如引用或 `const`），使得返回类型更精确。

7.8 函数模板与重载

- C++ 允许定义与函数模板同名的普通函数或其他模板。编译器根据参数匹配和重载解析规则，选择最合适的版本。

```
// 普通函数（优先匹配 int 类型）  
int Max(int a, int b) {  
    return a > b ? a : b;  
}  
  
template <typename T>  
T Max(T a, T b) {  
    return a > b ? a : b;  
}
```

7.8 函数模板与重载

- 具体来说，当普通函数和模板函数都可用时，编译器会优先选择**普通函数**，因为它与给定参数类型的匹配更精确。

```
// 普通函数（优先匹配 int 类型）
int Max(int a, int b) {
    return a > b ? a : b;
}

template <typename T>
T Max(T a, T b) {
    return a > b ? a : b;
}
```

7.8 函数模板与重载

- 如果你希望根据不同的参数类型定义不同版本的函数模板，还可以使用**模板重载**来实现。例如：

```
template <typename T>
T* Max(T* a, T* b) {
    return *a > *b ? a : b;
}
```

// 普通函数（优先匹配 `int` 类型）

```
int Max(int a, int b) {  
    return a > b ? a : b;  
}
```

```
template <typename T>  
T Max(T a, T b) {  
    return a > b ? a : b;  
}
```

```
template <typename T>  
T* Max(T* a, T* b) {  
    return *a > *b ? a : b;  
}
```

```
int main() {  
    int x{ 10 }, y{ 20 };  
    cout << Max(x, y) << '\n';  
    cout << *Max(&x, &y) << '\n';  
}
```

7.9 模板专门化

- 模板专门化（`Template Specialization`）允许我们为模板参数的特定类型或值提供专门的实现。

7.9.1 类型模板参数的专门化

- 由于传入的是 `int*` 类型，编译器会生成一个比较两个指针地址的版本：

```
#include <iostream>
template <typename T>
T Max(T a, T b) {
    return a > b ? a : b;
}

int main() {
    int x = 10, y = 20;
    int* p = &x, *q = &y;
    std::cout << *Max(p, q) << '\n';
}
```


7.9.1 类型模板参数的专门化

- 由于传入的是 `int*` 类型，编译器会生成一个比较两个指针地址的版本：

```
int* Max(int* a, int* b) {  
    return a > b ? a : b;  
}
```

7.9.1 类型模板参数的专门化

- 采用模板专门化为 `int*` 类型提供专门实现：

```
template <>
int* Max<int*>(int* a, int* b) {
    return *a > *b ? a : b;
}
```

使用 `template <>` 和 `functionName<SpecificType>` 来定义

7.9.2 非类型模板参数的专门化

- 同样，对于非类型模板参数也可以提供特定的模板实现。

```
// 通用模板：计算 N 的阶乘
template <int N>
struct Factorial {
    static const int value = N * Factorial<N - 1>::value;
};
```

```
// 专门化：N=0
template <>
struct Factorial<0> {
    static const int value = 1;
};
```

// 通用模板：计算 N 的阶乘

```
template <int N>
```

```
struct Factorial {
```

```
    static const int value = N * Factorial<N - 1>::value;
```

```
};
```

// 专门化：N=0

```
template <>
```

```
struct Factorial<0> {
```

```
    static const int value = 1;
```

```
};
```

// 专门化：N=1

```
template <>
```

```
struct Factorial<1> {
```

```
    static const int value = 1;
```

```
};
```

```
int main() {  
    std::cout << "0! = " << Factorial<0>::value << std::endl; // 输出 1  
    std::cout << "1! = " << Factorial<1>::value << std::endl; // 输出 1  
    std::cout << "5! = " << Factorial<5>::value << std::endl; // 输出 120  
    return 0;  
}
```

这个例子展示了非类型模板参数专门化的几个关键作用：

- **避免无限递归**

如果没有 $N=0$ 或 $N=1$ 的专门化，模板会一直递归下去（例如 `Factorial<-1>`），导致编译失败。专门化提供了终止条件，确保程序正常工作。

- **性能优化**

对于 `N=0` 和 `N=1`，直接返回结果而非递归计算，减少了编译时的开销。这种优化在模板元编程中尤为重要。

- **数学正确性**

阶乘的定义中， $0! = 1$ 和 $1! = 1$ 是边界条件。专门化保证了这些特殊情况被正确处理，符合数学规律。

7.10 可变模板参数

- **可变模板参数** (variadic templates) 是 C++11 引入的强大特性，允许模板接受任意数量的参数（类型或非类型），且这些参数可以类型各异，并在编译期处理。相比之下，`std::initializer_list` 仅支持同类型参数的任意数量集合，需运行时构造。

```
template<typename... Args>
void print(Args... args) {
    (std::cout << ... << args) << std::endl; // 折叠表达式 (C++17)
}
```

调用: `print(1, "hello", 3.14);` 输出: ``1hello3.14``

7.10.1. 什么是可变模板参数？

可变模板参数允许模板函数或类处理不定数量的参数。它们通过**参数包**表示，分为两种：

- **类型参数包**：表示零个或多个类型。
- **非类型参数包**：表示零个或多个非类型值（如整数、指针）。

7.10.1. 什么是可变模板参数?

- **类型参数包**: 表示零个或多个类型。

语法: `typename... Args` 或 `class... Args`。如:

```
// 类型参数包
template <typename... Args>
void print(Args... args) {}
```


7.10.1. 什么是可变模板参数?

- **类型参数包**: 表示零个或多个类型。

```
// 类型参数包
template <typename... Args>
void print(Args... args) {}
```

- `template <typename... Args>` 的 `Args` 表示模板参数包, 可以接受任意数量的模板参数。
- `void func(Args... args)` 的 `args` 是函数参数包, 表示函数接受多个参数。
- 调用 `print(1, "hello", 3.14)`, 参数包 `args` 会捕获这些参数。

7.10.1. 什么是可变模板参数?

- **非类型参数包**: 表示零个或多个非类型值 (如整数、指针)。

- 语法: `auto... values` (C++17 起) 或 `int... values`。如:

```
// 非类型参数包
template <int... values>
void print_values() {}
```

7.10.1. 什么是可变模板参数?

- **非类型参数包**: 表示零个或多个非类型值 (如整数、指针)。

```
// 非类型参数包  
template <int... values>  
void print_values() {}
```

- `values` 是一个非类型模板参数包, 它接受任意数量的常量整数。
- `print_values<1, 2, 3>()`, 参数包 `values` 会捕获这些参数。

```
#include <iostream>

template <typename... Args>
void print(Args... args) {
    std::cout << "Number of args: " << sizeof...(args) << "\n";
}

int main() {
    print();           // 0 个参数
    print(1);         // 1 个参数
    print(1, "hello", 3.14); // 3 个参数
}
```

输出:

```
Number of args: 0
Number of args: 1
Number of args: 3
```

`sizeof...(args)`: 返回参数包的元素个数, 编译期常量。

7.10.2 参数包展开

- **参数包展开**是最基础的可变模板参数处理方式，指将参数包（**pack...**）的元素逐个代入某个模式（**pattern**），生成一系列值、表达式或参数。它是 C++11 引入的核心机制，贯穿几乎所有参数包操作。

7.10.2 参数包展开

- 语法：
 - `pack...`：直接展开参数包的元素。
 - `pattern(pack)...`：将每个元素代入模式，生成多个结果。
 - `pack..., expr`：展开参数包并附加其他值或表达式。
- 机制：
 - 编译器将参数包的每个元素“复制”到指定模式，形成展开后的序列。
 - 展开结果可以是值（如初始化列表）、表达式（如函数调用）或模板参数。

7.10.2 参数包展开

- 特点：
 - 生成**多个**值或表达式（区别于折叠表达式的单一值）。
 - 不需要操作符，灵活性高。
 - 用途广泛：初始化容器、传递参数、构造模板实例等。

示例 1: 初始化数组

```
#include <array>
#include <iostream>

template <auto... Values>
constexpr auto make_array() {
    return std::array{Values...}; // 展开为初始化列表
}

int main() {
    auto arr = make_array<1, 2, 3, 4>();
    for (auto x : arr) {
        std::cout << x << " "; // 输出: 1 2 3 4
    }
}
```


示例 1: 初始化数组

```
#include <array>
#include <iostream>

template <auto... Values>
constexpr auto make_array() {
    return std::array{Values...}; // 展开为初始化列表
}

int main() {
    auto arr = make_array<1, 2, 3, 4>();
    for (auto x : arr) {
        std::cout << x << " "; // 输出: 1 2 3 4
    }
}
```

```
template <auto... Values>
constexpr auto make_array() {
    return std::array{Values...}; // 展开为初始化列表
}
```

```
make_array<1, 2, 3, 4>();
```

分析：

- `Values...` 是非类型参数包，包含 `1, 2, 3, 4`。
- `std::array{Values...}` 展开为 `std::array<int, 4>{1, 2, 3, 4}`。
- 展开模式是“初始化列表项”，每个 `Values` 元素成为数组的一个元素。|

示例 2：函数参数传递

```
#include <iostream>

template <typename... Args>
void forward_args(Args... args) {
    process(args...); // 展开 args... 传递给 process
}

void process(int x, double y, const char* z) {
    std::cout << x << " " << y << " " << z << "\n";
}

int main() {
    forward_args(42, 3.14, "hello"); // 输出: 42 3.14 hello
}
```

```
template <typename... Args>
void forward_args(Args... args) {
    process(args...); // 展开 args... 传递给 process
}

void process(int x, double y, const char* z) {
    std::cout << x << " " << y << " " << z << "\n";
}

int main() {
    forward_args(42, 3.14, "hello"); // 输出: 42 3.14 hello
}
```

- `args...` 是类型参数包，捕获 `42` (`int`)、`3.14` (`double`)、`"hello"` (`const char*`)
- `process(args...)` 展开为 `process(42, 3.14, "hello")`。
- 展开模式是“函数调用参数”，每个 `args` 元素成为 `process` 的一个实参。|

7.10.3 折叠表达式 (Fold Expression)

- **折叠表达式**是 C++17 引入的一种特性，用于对参数包 (parameter pack) 的元素通过一个二元操作符 (如 `+`, `*`, `&&`, 等) 进行组合，最终生成一个单一值。它是参数包展开的一种形式，但目的不在于生成多个表达式，而是聚合为一个结果。

四种形式：

类型	语法形式	示例	含义
一元右折叠	<code>(pack op ...)</code>	<code>(args + ...)</code>	从左至右组合
一元左折叠	<code>(... op pack)</code>	<code>(... + args)</code>	从右至左组合
二元右折叠	<code>(pack op ... op init)</code>	<code>(args + ... + 0)</code>	类似 reduce，末尾附初始值
二元左折叠	<code>(init op ... op pack)</code>	<code>(0 + ... + args)</code>	从初始值开始依次组合参数

示例 1: 求和

```
#include <iostream>

template <typename... Args>
auto sum(Args... args) {
    return (args + ...); // 一元右折叠
}

int main() {
    std::cout << sum(1, 2, 3, 4) << "\n"; // 输出: 10
    std::cout << sum() << "\n";           // 错误: 一元折叠空包未定义
}
```

```
template <typename... Args>
auto sum(Args... args) {
    return (args + ...); // 一元右折叠
}
```

函数模板实例化说明

调用 `sum(1, 2, 3, 4)` 时，模板参数自动推导为：

```
auto sum(int, int, int, int);
```

- 模板参数包 `Args...` 推导为 `{int, int, int, int}`。
- 函数参数包 `args...` 绑定到实参 `{1, 2, 3, 4}`。

```
template <typename... Args>
auto sum(Args... args) {
    return (args + ...); // 一元右折叠
}
```

```
sum(1, 2, 3, 4)
```

函数体中的折叠表达式：

```
return (args + ...);
```

- `args` 是参数包，包含多个值。
- `+` 是用于折叠的操作符。
- `...` 是折叠表达式的展开符。
- `()` 表示折叠范围，确保语法完整。


```
template <typename... Args>
auto sum(Args... args) {
    return (args + ...); // 一元右折叠
}
```

一元右折叠的展开规则 (C++17)

当参数包包含元素 `E1, E2, ..., EN` ($N \geq 1$) 时:

`(args + ...)` // 展开为: `((E1 + E2) + E3) + ... + EN`

- 操作顺序是从左到右 (尽管形式叫“右折叠”, 但展开结果在加法下等价于左折叠)。
- 如果参数包仅有一个元素 `(E1)`, 则折叠结果为 `E1` 本身。
- 如果参数包为空 ($N = 0$), 一元折叠无法展开, 编译错误。

`((1 + 2) + 3) + 4`

```
template <typename... Args>
auto sum(Args... args) {
    return (args + ...); // 一元右折叠
}
```

```
std::cout << sum(1, 2, 3, 4)
std::cout << sum() << "\n";
```

对空包的处理

为了支持空参数包，可以使用 **二元右折叠**，添加初始值以提供“默认结果”：

```
return (args + ... + 0); // 空包时返回 0
```

此时的折叠展开为：

```
((E1 + E2) + ...) + 0
```

当没有任何参数时，将直接返回初始值 `0`，避免编译错误。

示例 2：顺序打印参数

```
#include <iostream>

template <typename... Args>
void print(Args... args) {
    ((std::cout << args << " "), ...); // 折叠表达式（逗号操作符）
}

int main() {
    print(1, "hello", 3.14); // 输出: 1 hello 3.14
}
```

- `args = 1, "hello", 3.14`。
- `((std::cout << args << " "), ...)`

展开为：

- `std::cout << 1 << " ", std::cout << "hello" << " ", std::cout << 3.14 << " "`
- 逗号操作符确保顺序执行，每个表达式返回 `std::ostream&`，最终折叠为单一值（流对象）。

7.10.3 递归模板处理

递归模板处理：一种简单方法，把一堆参数拆开，逐个处理！

- 想象你有一堆东西要处理（比如数字、字符串），递归模板就像把这堆东西分成“第一个”和“剩下的”，先处理第一个，再把剩下的继续拆分，直到处理完为止。这在 C++11/14 是处理可变模板参数的主要方式，C++17 后有了更简单的折叠表达式，但递归依然很实用。

7.10.3 递归模板处理

怎么做？

- **基础情况：**当只剩一个或没参数时，直接处理（比如返回结果或打印）。
- **递归情况：**处理第一个参数，然后把剩下的参数交给同一个函数继续处理。
- **工作原理：**每次从参数包里拿出一个，剩下的继续递归，直到全部处理完。

7.10.3 递归模板处理

语法：

- 定义基础情况（base case）：处理空包或单一参数。
- 定义递归情况（recursive case）：处理第一个元素，递归剩余参数。

7.10.3 递归模板处理

求和

// 基情形：当只有一个参数时，直接返回该参数

```
int sum(int a) {  
    return a;  
}
```

// 递归情形：将第一个参数与余下参数的和相加

```
template <typename... Args>  
int sum(int first, Args... args) {  
    return first + sum(args...); // 递归调用，将 args... 中的所有参数累加  
}
```

// 基情形：当只有一个参数时，直接返回该参数

```
int sum(int a) {  
    return a;  
}
```

// 递归情形：将第一个参数与余下参数的和相加

```
template <typename... Args>  
int sum(int first, Args... args) {  
    return first + sum(args...); // 递归调用，将 args... 中的所有参数累加  
}
```

工作过程说明：

- 当调用 `sum(1, 2, 3)` 时：
 1. `sum(1, 2, 3)` 拆分为 `1 + sum(2, 3)`。
 2. `sum(2, 3)` 拆分为 `2 + sum(3)`。
 3. 当只剩下一个参数时，调用基情形 `sum(3)`，直接返回 `3`。
 4. 最终展开为 `1 + (2 + 3)`，得到 `6`。

递归打印所有参数

```
template <typename T>
void print(T t) {
    std::cout << t << std::endl;
}
```

```
template <typename T, typename... Rest>
void print(T t, Rest... rest) {
    std::cout << t << " ";
    print(rest...); // 递归打印剩余参数
}
```

```
int main() {
    print("Hello,", "world!", 123, 4.56);
    // 输出: Hello, world! 123 4.56
    return 0;
}
```

7.10.3 递归模板处理

适合场景

- 想一个个处理参数，比如打印每个参数。
- 需要把参数累加或组合，比如求和、生成列表。
- 复杂任务，比如需要条件判断或编译期计算。

参数包展开、折叠表达式和递归模板处理的比较

特性	参数包展开	折叠表达式	递归模板处理
引入版本	C++11	C++17	C++11
输出	多个值/表达式	单一值	任意（序列或单一值）
灵活性	高（多种模式）	中（限于操作符）	最高（支持复杂逻辑）
代码简洁性	简洁	极简	较复杂
编译开销	低	低	高（递归实例化）
调试难度	中（展开错误晦涩）	低（语法简单）	高（递归错误复杂）
空包处理	无需特化	一元折叠需检查，二元安全	需显式基础情况
适用场景	初始化、参数传递	聚合计算（求和、逻辑）	复杂逻辑、序列生成
在 generate_square_table 中	扩展 Values...、初始化数组	不适用（需序列）	核心逻辑（控制范围）

参数包展开、折叠表达式和递归模板处理的比较

优点对比

- **参数包展开**：最通用，适合序列生成，代码简洁，编译高效。
- **折叠表达式**：最简洁，聚合场景无敌，空包处理（二元）方便。
- **递归模板处理**：逻辑最灵活，适合复杂任务，能生成序列或单一值。

缺点对比

- **参数包展开**：无逻辑控制，需其他方式配合，复杂场景受限。
- **折叠表达式**：仅单一值，操作符限制，复杂逻辑需额外处理。
- **递归模板处理**：代码长，编译慢，调试难，简单任务显得冗余。

7.10.5 编译期平方表生成器

- 见书：《现代C++编程实战：从入门到用应用》

关注

Youtube频道: <https://www.youtube.com/c/hwdong>

博客: <https://hwdong-net.github.io>